Name: Gunjan Rupani
Batch: T22
Roll no: 2401096
Subject:ASD&D

Experiment No. 07

Aim: To implement monitoring using Prometheus / Grafana / Nagios

LO Mapping: LO5

LO5: Implement ML lifecycle management and automated workflows using MLflow, Airflow, and monitoring tools like Prometheus/Grafana/Nagios.

Theory:

Continuous monitoring is a process to detect, report, respond all the attacks which occur in its infrastructure. Once the application is deployed into the server, the role of continuous monitoring comes in to play. The entire process is all about taking care of the company's infrastructure and respond appropriately.

Why We Need Nagios tool?

Here, are the important reasons to use Nagios monitoring tool:

- Detects all types of network or server issues
- Helps you to find the root cause of the problem which allows you to get the permanent solution to the problem
- Active monitoring of your entire infrastructure and business processes
- Allows you to monitor and troubleshoot server performance issues
- Helps you to plan for infrastructure upgrades before outdated systems create failures
- You can maintain the security and availability of the service
- Automatically fix problems in a panic situation
- Nagios is the most popular, open source, powerful monitoring system for any kind of infrastructure. It enables organizations to identify and resolve IT infrastructure problems before they affect critical business processes. Nagios has the capability of monitoring applications, services and entire IT infrastructure.
- NRPE is known as Nagios Remote Plugin Executor. It is designed to execute plugins on remote systems. On Windows hosts, the equivalent remote agent is NSClient++, which listens for Nagios requests and executes local checks (disk, CPU, memory, services, etc.).
- NSClient++ runs as a Windows service on the remote host and waits for requests from the Nagios server (usually over the NRPE protocol, which NSClient++ also supports). When the Nagios server

needs to check a resource on that host, it sends a command; NSClient++ executes the check locally and returns the result to Nagios.

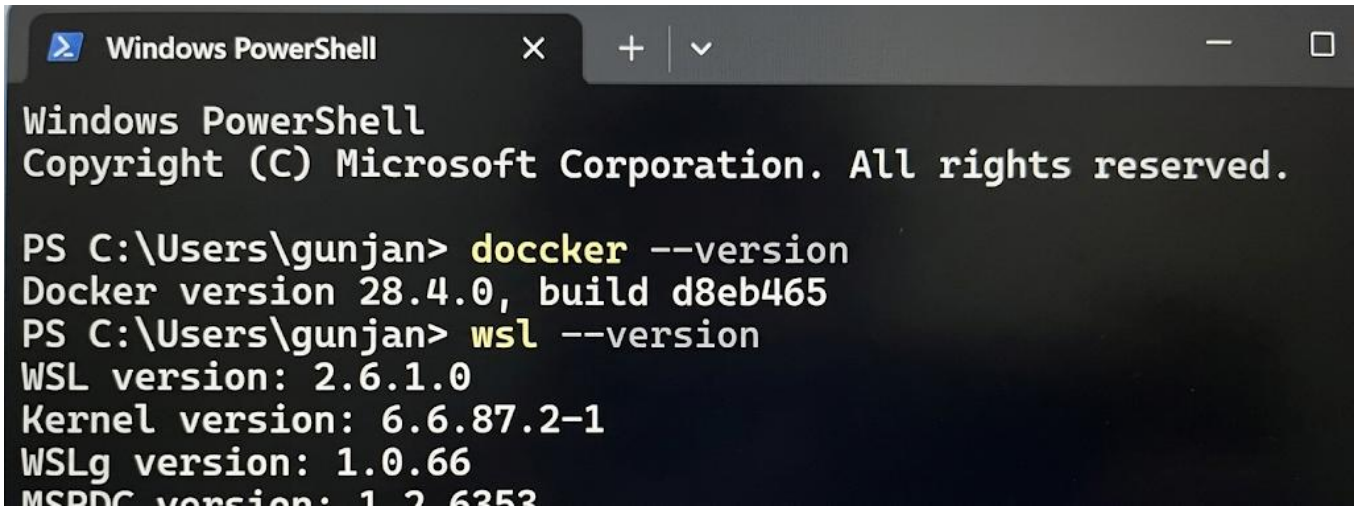
Note on running Nagios on Windows

Nagios Core is built for Linux and does not install natively on Windows. The standard practice for a Windows machine is to run the Nagios Core server inside a Docker container (using Docker Desktop with the WSL2 backend), and to monitor Windows hosts using NSClient++, which is the Windows counterpart of NRPE. All commands below are Windows PowerShell / Command Prompt commands.

Output:

Step 1: Verify Docker and WSL2

The Docker and WSL2 installations were verified using:



Step 2: Pull Nagios Docker Image

The Nagios Docker image was downloaded using:

```
PS C:\Users\gunjan> docker pull jasonrivers/nagios:latest
latest: Pulling from jasonrivers/nagios
fdbaa15b8986: Pull complete
1413dee23aae: Pull complete
50ea270d34e9: Pull complete
dc3a88012a3e: Pull complete
cb259a83ac3d: Pull complete
1b9771f55a12: Pull complete
37f75c8146e2: Pull complete
f47c3af6bf44: Pull complete
7a7447982198: Pull complete
80b6ab7aef5d: Pull complete
d003d4303177: Pull complete
c42073aaf047: Pull complete
a80db686d641: Pull complete
a6b90b5bbb15: Pull complete
0d59a81a5608: Pull complete
6fe99ef1bed3: Pull complete
a03720c2c16b: Pull complete
e4dc1d4e010c: Pull complete
9288c4992679: Pull complete
c5701b5e1540: Pull complete
e68fcda12c0c: Pull complete
254354b70ac0: Pull complete
2908452ff193: Pull complete
54b9e993e315: Pull complete
a7bd6dce6a77: Pull complete
748b632e75e7: Pull complete
7edde1931fd2: Pull complete
```

The image was successfully downloaded.

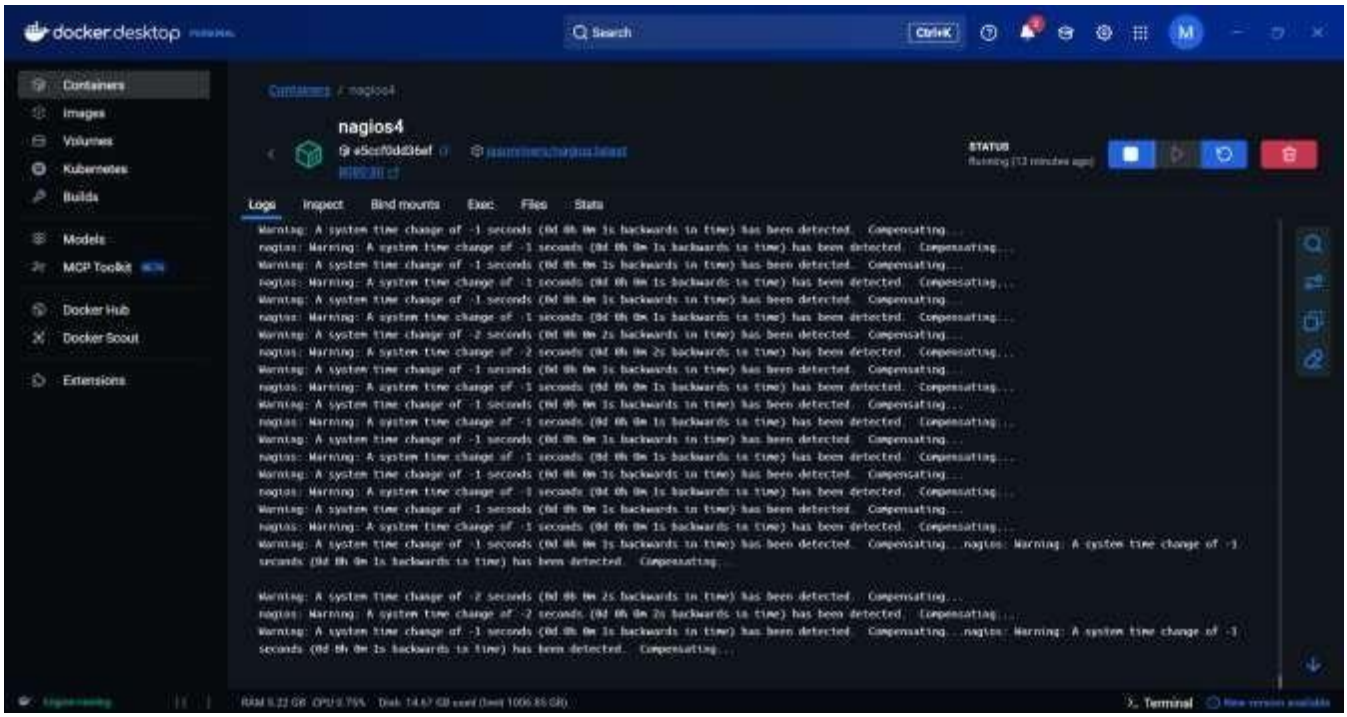
Then the Nagios container was created using:

The container was verified using:

```
CONTAINER ID   IMAGE                     COMMAND                  CREATED         STATUS         PORTS                                       NAMES
e5ccf0dd36ef   jasonrivers/nagios:latest  "/usr/local/bin/star…"   27 seconds ago  Up 26 seconds  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp     nagios4
```

Step 3: Configure Nagios Admin Password

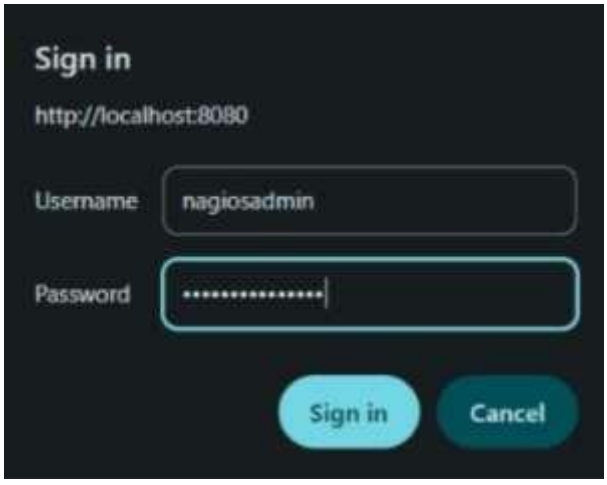
The Nagios administrator password was configured using:

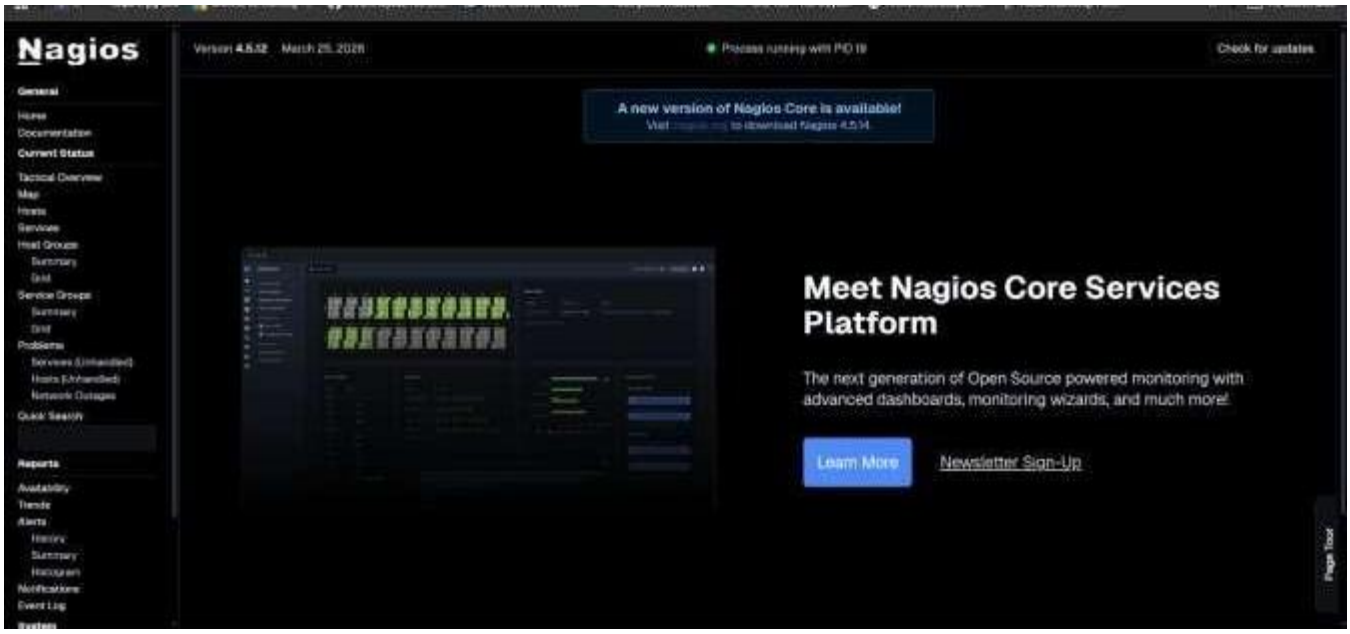


Step 4: Access Nagios Web Interface

Nagios was accessed through: http://localhost:8080/nagios/

The administrator credentials were used to log in.





Step 5: Install and Configure NSClient++

NSClient++ was installed on the Windows system to act as the Windows monitoring agent.

The NSClient++ service was verified using:

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Windows\system32> Get-Service nscp

Status   Name               DisplayName
------   ----               -----------
Running  nscp               NSClient++ Monitoring Agent
```

Step 6: Verify NSClient++ Monitoring Port NSClient++

uses port **5666** for NRPE communication.

The port was verified using:

```
PS C:\Windows\system32> netstat -ano | findstr :5666
  TCP    0.0.0.0:5666           0.0.0.0:0              LISTENING       28164
  TCP    [::]:5666              [::]:0                 LISTENING       28164
```

Step 7: Configure Windows Host in Nagios A

Nagios configuration file was created:

/opt/nagios/etc/servers/MyWindowsHost001.cf

g

The Windows host was configured with:

Host Name: Windows_Host_001

Alias: Windows Host 001

Because direct PING/NRPE connectivity to the Windows LAN address was unavailable from the Docker network, the monitoring endpoint was configured using:

host.docker.internal and

NSClient++ port:

5666

This allowed Nagios to monitor the Windows host through the reachable NSClient++ endpoint.

Step 8: Validate Nagios Configuration

The Nagios configuration was checked using:

```
PS C:\Windows\system32> docker exec -it nagios4 /opt/nagios/bin/nagios -v /opt/nagios/etc/nagios.cfg

Nagios Core 4.5.12
Copyright (c) 2009-present Nagios Core Development Team and Community Contributors
Copyright (c) 1999-2009 Ethan Galstad
Last Modified: 2026-03-25
License: GPL

Website: https://www.nagios.org
Reading configuration data...
   Read main config file okay...
   Read object config files okay...

Running pre-flight check on configuration data...

Checking objects...
Warning: Service 'PING' on host 'Windows_Host_001' has no default contacts or contactgroups defined!
        Checked 8 services.
Warning: Host 'Windows_Host_001' has no default contacts or contactgroups defined!
        Checked 2 hosts.
        Checked 1 host groups.
        Checked 0 service groups.
        Checked 1 contacts.
        Checked 1 contact groups.
        Checked 23 commands.
        Checked 5 time periods.
        Checked 0 host escalations.
        Checked 0 service escalations.
Checking for circular paths...
        Checked 2 hosts
        Checked 0 service dependencies
        Checked 0 host dependencies
        Checked 5 timeperiods
Checking global event handlers...
Checking obsessive compulsive processor commands...
Checking misc settings...

Total Warnings: 2
Total Errors:   0

Things look okay - No serious problems were detected during the pre-flight check
```

Step 9: Restart Nagios

After configuration, the Nagios container was restarted using:

```
PS C:\Windows\system32> docker restart nagios4
nagios4
```

Step 10: Verify Host in Nagios

The Nagios web interface was opened again:

http://localhost:8080/nagios/

Then:

Current Status → Hosts



Results

Nagios Core was successfully deployed inside a Docker container on Windows. NSClient++ was configured as the Windows monitoring agent, and the Windows host was successfully added to Nagios.

The final Nagios dashboard showed: Windows_Host_001

→ UP

with a successful TCP monitoring response from the NSClient++ monitoring port.

Conclusion:

Thus, monitoring was successfully implemented using Nagios Core running in Docker. NSClient++ was configured on the Windows system as the monitoring agent, and the Windows host was successfully added and monitored through the Nagios web interface. The final monitoring status of the configured host was verified as UP.